



OASIS Security Report

Server-Side Request Forgery Security Analysis

Date: 2026-06-12 01:31:23 — **Model:** llama3.2:3b

Vulnerability: Server-Side Request Forgery

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **9** file(s) for Server-Side Request Forgery.

[Return to table of contents](#)

Statistics

Files analyzed: 9 **Total findings:** 11

Critical



10

High		1
Medium		0
Low		0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.683
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.680
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.676
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.667
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.654
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.639
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.638
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.638
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.622

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.683

Source lines: 1-85

► Finding 1.1: Vulnerability found (Critical)

Vulnerable code

```
function CashOut(uint _am) public payable{}
if(_am<=balances[msg.sender])
{
    // <yes> <report> REENTRANCY
    if(msg.sender.call.value(_am)())
    {
        balances[msg.sender]-=_am;
        TransferLog.AddMessage(msg.sender,_am,"CashOut");
    }
}
```

Explanation

The `call` method is vulnerable to Server-Side Request Forgery because it allows the attacker to execute arbitrary code on behalf of the contract.

Impact

Access to internal services, data theft, or system compromise via internal network

Entry point

/CashOut(uint)

Execution path

```
call msg.sender.value(_am)()
```

HTTP methods

POST

Parameters

_am

Exploitation steps

- Attacker crafts a malicious call with a large value to drain the contract's balance.

Example payloads

- `_am = 1e18;`

Conditions

Contract owner has sufficient balance and is authenticated.

Remediation

Use safer methods like ``transfer`` instead of ``call``.

Secure example

```
function CashOut(uint _am) public payable {  
    if (_am <= balances[msg.sender]) {  
        balances[msg.sender] -= _am;  
        TransferLog.AddMessage(msg.sender, _am, "CashOut");  
    }  
}  
cashOut = transfer(_amount);
```

File 2: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.680

Source lines: 1-35

► Finding 2.1: Server-Side Request Forgery (Critical)

Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

Explanation

The call() function is used without proper validation or sanitization, allowing an attacker to induce the server to make requests with arbitrary parameters.

Impact

Access to internal services and data theft via internal network

Entry point

/withdraw(uint)

HTTP methods

POST

Parameters

amount, to

Exploitation steps

- Attacker crafts a malicious amount or to value

Example payloads

- `0x1234567890123456789012345678901234567890`

HTTP raw requests (Burp / ZAP)

```
POST /withdraw(uint) HTTP/1.1\r\nHost: 127.0.0.1\r\nContent-Length: 0\r\n\r\n
```

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.676

Source lines: 1-39

► Finding 3.1: Server-Side Request Forgery (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

Explanation

The contract calls msg.sender's external function without validating the output, allowing an attacker to make requests on behalf of the sender.

Impact

Access to internal services, data theft, or system compromise via internal network

Entry point

/withdrawFunds

HTTP methods

POST

Parameters

_weiToWithdraw, msg.sender

Exploitation steps

- The attacker sends a request with _weiToWithdraw set to a large value.
- The contract calls the external function without validating its output.
- The attacker manipulates the output of the external function to make requests on behalf of the sender.

Example payloads

- large amount of ether

Conditions

msg.sender has access to the contract's internal functions.

Remediation

Validate and sanitize the output of the external function call.

Secure example

```
require(msg.sender.call.value(_weiToWithdraw).send())
```

File 4: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.667

Source lines: 1-28

► Finding 4.1: Vulnerability found (Critical)

Vulnerable code

```
function solve(string solution) public {  
    require(hash == sha3(solution));  
    msg.sender.transfer(1000 ether);  
}
```

Explanation

The contract allows the sender of the request to receive 1000 ether without proper validation or sanitization, making it vulnerable to Server-Side Request Forgery.

Impact

Access to internal services, data theft, or system compromise via internal network

Entry point

/transfer

HTTP methods

POST

Parameters

solution

Exploitation steps

- The attacker sends a request to the contract with a solution that hashes to the expected hash.
- The contract processes the request and transfers 1000 ether to the sender.

Example payloads

- hashing-specific-input

Conditions

attacker can reach POST /transfer while authenticated

Remediation

Validate and sanitize user input, use allowlists for approved solutions.

Secure example

```
require(hash == sha3(solution));  
msg.sender.transfer(1000 ether);
```

File 5: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.654

Source lines: 1-31

► Finding 5.1: Vulnerability found (Critical)

Vulnerable code

```
function sendTo(address receiver, uint amount) public {  
    require(tx.origin == owner);  
    receiver.transfer(amount);  
}
```

Explanation

The 'tx.origin' variable is used for authorization without validation.

Impact

Access to internal services, data theft or system compromise via internal network

Entry point

/transfer

HTTP methods

POST

Parameters

receiver, amount

Exploitation steps

- An attacker can make the 'owner' send funds to another address.

Example payloads

- tx.origin: '0x742d35Cc6634C0532925a3b844Bc454e4438f44E'

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1  
Host: 0x742d35Cc6634C0532925a3b844Bc454e4438f44E  
Content-Length: 0
```

File 6: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.639

Source lines: 1-61

► Finding 6.1: Vulnerability found (Critical)

Vulnerable code

```
players[tot] = Player(msg.sender, number); tot++;
```

Explanation

This line allows an attacker to send arbitrary requests by manipulating the `number` variable.

Impact

Access to internal services and data theft via internal network

Entry point

/play

HTTP methods

POST

Parameters

number

Exploitation steps

- Manipulate the `number` variable in the request payload to bypass checks

Example payloads

- "1"

Conditions

Authenticated user with malicious intent

Remediation

Validate and sanitize the `number` input before using it

Secure example

```
players[tot] = Player(msg.sender, uint256(number)); tot++;
```

File 7: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.638

Source lines: 1-42

► Finding 7.1: Vulnerability found (Critical)

Vulnerable code

```
owner.transfer(reward);
```

Explanation

The `transfer()` function is used to send Ether from the contract, but it does not check if the recipient exists. An attacker can exploit this by setting `msg.sender` to a fake address.

Impact

Access to internal services via fake transfer requests

Entry point

/transfer

HTTP methods

POST

Parameters

msg.sender

Exploitation steps

- Set `msg.sender` to a fake address

Conditions

Authenticated users can make transfers.

Remediation

Use the `call()` function instead of `transfer()` to prevent direct Ether transfer

Secure example

```
owner.call{  
    value: reward  
}()
```

File 8: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.638

Source lines: 126-467

► Finding 8.1: Server-Side Request Forgery (Critical)

Vulnerable code

```
function _walletLibrary.delegatecall(msg.data); //it should have whitelisted
specific methods that the user is allowed to call
```

Explanation

The `\_walletLibrary.delegatecall` function allows an attacker to induce a request by passing malicious data, bypassing input validation.

Impact

An attacker can force the contract to make requests without authentication or authorization.

Entry point

`_walletLibrary.delegatecall(msg.data)`

HTTP methods

POST

Parameters

`msg.data`

Exploitation steps

- Pass malicious data as `msg.data`
- Delegate to `\_walletLibrary(delegatecall(...))` with malicious data

Example payloads

- `"http://malicious.com/endpoint"`
- `"POST /transfer HTTP/1.1 Host: malicious.com
Content-Type: application/x-www-form-urlencoded

malicious_data&=foo"`

Conditions

Attacker can reach POST /transfer while authenticated.

Remediation

Validate and sanitize `msg.data` before passing it to `\_walletLibrary.delegatecall`. Implement proper authentication and authorization mechanisms.

Source lines: 207-477

► Finding 8.1: Server-Side Request Forgery (Critical)

Vulnerable code

```
_walletLibrary.delegatecall(msg.data); //it should have whitelisted specific methods that the user is allowed to call
```

Explanation

This delegatecall allows an attacker to induce the server to make requests by passing a malicious `msg.data` payload.

Impact

Access to internal services, data theft, or system compromise via internal network

Entry point

/

Execution path

```
delegatecall(msg.data) -> check if delegatecall is whitelisted -> execute delegatecall
```

Parameters

msg.data

Exploitation steps

- Attacker crafts a malicious msg.data payload
- Attacker sends the malicious payload to the contract

Example payloads

- `https://example.com/evilpayload`

HTTP raw requests (Burp / ZAP)

```
POST / HTTP/1.1
Host: example.com

GET https://example.net/evil
```

File 9: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.622

Source lines: 1-286

► Finding 9.1: Vulnerable Code (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

The `execute` function directly calls the `\_to` contract without any validation on its contents.

Impact

Allowing an attacker to execute arbitrary code in the victim's contract.

Entry point

/execute

HTTP methods

POST

Parameters

_to, _value, _data

Exploitation steps

- Send a malicious `\_to` value with a large amount of gas
- Include a malicious `\_data` payload

Example payloads

- malicious contract address

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: <contract_address>
Content-Length: 0
```

Source lines: 96-414

► Finding 9.1: Server-Side Request Forgery (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner  
returns (bytes32 o_hash)
```

Explanation

The function `execute` allows an attacker to induce the server to make requests by passing a malicious URL as part of the `\_data` bytes.

Impact

An attacker can execute arbitrary code on the server, potentially leading to data theft or system compromise.

Entry point

/execute

HTTP methods

POST

Parameters

_data

Exploitation steps

- Pass a malicious URL as part of the `\_data` bytes to induce the server to make requests.

Example payloads

- malicious URL
- malicious payload

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1  
Host: example.com  
Content-Type: application/json  
  
{"url": "/malicious-url"}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)